PSVM: Parallel Support Vector Machines

Edward Y. Chang, Kaihua Zhu, Hang Cui, Hongjie Bai, Jian Li, Zhihuan Qiu, and Hao Wang EDCHANG@GOOGLE.COM
Google Research, Beijing, China

Abstract

Support Vector Machines (SVMs) suffer from a widely recognized scalability problem in both memory use and computational time. To improve scalability, we have developed a parallel SVM algorithm (PSVM), which reduces memory use through performing a row-based, approximate matrix factorization, and which loads only essential data to each machine to perform parallel computation. Let n denote the number of training instances, p the reduced matrix dimension after factorization (p is significantly smaller than n), and m the number of machines. PSVM reduces the memory requirement from $\mathcal{O}(n^2)$ to $\mathcal{O}(np/m)$, and improves computation time to $\mathcal{O}(np^2/m)$.

1. Introduction

Support Vector Machines (SVMs) are a core machine learning technology. They enjoy strong theoretical foundations and excellent empirical successes in many pattern recognition applications. Unfortunately, SVMs do not scale well with respect to the size of training data. Given n training instances, the time to train an SVM model is about $\mathcal{O}(n^2)$ in the average case, and so is the memory requirement. These excessive costs make SVMs impractical for large-scale applications.

Let us examine the resource bottlenecks of SVMs in a binary classification setting to explain our proposed solution. Given a set of training data $\mathcal{X} = \{(\mathbf{x}_i, y_i)|\mathbf{x}_i \in \mathbf{R}^d\}_{i=1}^n$, where $\mathbf{x}_i$ is an observation vector, $y_i \in \{-1, 1\}$ is the class label of $\mathbf{x}_i$, and n is the size of $\mathcal{X}$, we apply SVMs on $\mathcal{X}$ to train a binary classifier. SVMs aim to search a hyperplane in the *Reproducing Kernel Hilbert Space* (RKHS) that maximizes the margin between the two classes of data in $\mathcal{X}$ with the smallest training error (Vapnik, 1995). This problem can be formulated as the following quadratic optimization problem:

$$\begin{aligned} \min \quad & \mathcal{P}(\mathbf{w}, b, \boldsymbol{\xi}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i \\ s.t. \quad & 1 - y_i(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_i) + b) \leq \xi_i, \quad \xi_i > 0, \end{aligned} \tag{1}$$

where $\mathbf{w}$ is a weighting vector, b is a threshold, C a regularization hyperparameter, and $\boldsymbol{\phi}(\cdot)$ a basis function which maps $\mathbf{x}_i$ to an RKHS space. The decision function of SVMs is $f(\mathbf{x}) = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + b$, where the $\mathbf{w}$ and b are attained by solving $\mathcal{P}$ in (1). The optimization problem in (1) is called the primal formulation of 2-norm SVMs with L_1 loss. It is hard to solve $\mathcal{P}$ directly, partly because the explicit mapping via $\boldsymbol{\phi}(\cdot)$ can make the problem intractable and partly because the mapping function $\boldsymbol{\phi}(\cdot)$ is often unknown. The method of *Lagrangian multipliers* (Courant & Hilbert, 1953) is thus introduced to transform the primal formulation into the dual one

$$\begin{aligned} \min \quad & \mathcal{D}(\boldsymbol{\alpha}) = \frac{1}{2}\boldsymbol{\alpha}^T\mathbf{Q}\boldsymbol{\alpha} - \boldsymbol{\alpha}^T\mathbf{1} \\ s.t. \quad & \mathbf{0} \leq \boldsymbol{\alpha} \leq \mathbf{C}, \ \mathbf{y}^T\boldsymbol{\alpha} = 0, \end{aligned} \tag{2}$$

where $[\mathbf{Q}]_{ij} = y_i y_j \boldsymbol{\phi}^T(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_j)$, and $\boldsymbol{\alpha} \in \mathbf{R}^n$ is the Lagrangian multiplier variable (or dual variable). The weighing vector $\mathbf{w}$ is related with $\boldsymbol{\alpha}$ in $\mathbf{w} = \sum_{i=1}^n \alpha_i\boldsymbol{\phi}(\mathbf{x}_i)$.

The dual formulation $\mathcal{D}(\boldsymbol{\alpha})$ requires an inner product of $\boldsymbol{\phi}(\mathbf{x}_i)$ and $\boldsymbol{\phi}(\mathbf{x}_j)$. SVMs utilize the *kernel trick* (Aizerman et al., 1964) by specifying a kernel function to define the inner-product $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}^T(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_j)$. We thus can rewrite $[\mathbf{Q}]_{ij}$ as $y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$. The only requirement for the kernel function K is that it must be positive semi-definite (psd), which theoretically guarantees that the RKHS space is mathematically valid (Vapnik, 1995). When the given kernel function K is psd, the dual problem $\mathcal{D}(\boldsymbol{\alpha})$ is a convex Quadratic Programming (QP) problem with linear constraints, which can be solved via the *Interior-Point method* (IPM) (Mehrotra, 1992). Both the computational and memory bottlenecks of the SVM training is the IPM solver to the dual formulation of SVMs in (2).

Currently, the most effective IPM algorithm is the primal-dual IPM (Mehrotra, 1992). The principal idea of the primal-dual IPM is to remove inequality constraints using a barrier function and then resort to the iterative Newton's

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

method to solve the KKT linear system related to the Hessian matrix $\mathbf{Q}$ in $\mathcal{D}(\boldsymbol{\alpha})$. The computational cost is $O(n^3)$ and the memory usage $O(n^2)$. Faster sampling-based and decomposition-based algorithms (discussed in Section 2) have recently been developed to reduce the memory use and the computational cost. However, their training time on large datasets is still quite excessive. For example, on a Pentium 2.8G-HZ machine with 8G-bytes RAM, the fastest algorithm is SVMLight (Joachims, 1998) with a finely tuned working-set size, which takes about six days to complete training on a one-million binary-class dataset.

In this paper, we propose a parallel SVM algorithm (PSVM) to reduce memory use and to parallelize both data loading and computation. Given n training instances each with d dimensions, PSVM first loads the training data in a round-robin fashion onto m machines. The memory requirement per machine is $\mathcal{O}(nd/m)$. Next, PSVM performs a parallel row-based Incomplete Cholesky Factorization (ICF) on the loaded data. At the end of parallel ICF, each machine stores only a fraction of the factorized matrix, which takes up space of $\mathcal{O}(np/m)$, where p is the column dimension of the factorized matrix. In a large-scale problem where n is very large, p is typically larger than d. PSVM effectively reduces memory use from $\mathcal{O}(n^2)$ to $\mathcal{O}(np/m)$, where p/m is much smaller than n. PSVM then performs parallel IPM to solve the quadratic optimization problem in (2). The computation time is improved from about $\mathcal{O}(n^2)$ of a decomposition-based algorithm (e.g., SVMLight, LIBSVM, and SMO) to $\mathcal{O}(np^2/m)$. Our experiments show that a six-day training task by a well-tuned SVMLight, takes PSVM just four hours to complete on $m = 20$ machines; and a seven-hour task by LIBSVM, takes PSVM 40 minutes on $m = 7$ machines.

Our contributions can be summarized as follows:

1. We propose PSVM, which achieves memory reduction and computation speedup via a novel parallel ICF algorithm and parallel IPM.
2. PSVM can adjust its p and m according to both time and resource constraints. Typically, p can be set to be about $\sqrt{n}$ without noticeably degrading training accuracy.
3. We have implemented PSVM in a production environment, and it has been used by several internal projects for classifying Web documents and images. PSVM bears out rigorous testing.

2. Related Work

Related work in speeding up SVMs can be categorized into two approaches: *data-processing* and *algorithmic*. The *data-processing* approach focuses on training-data selection to reduce n. When n is very large, however, we must still resort to the *algorithmic* approach, which devises approximate algorithms to speed up the QP solver. These two approaches can be complementary to each other. The algorithmic approach can be further divided into two methods: *decomposition* and *factorization*. Both methods directly aim to speed up solving the QP problem in (2).

The decomposition-based method divides a large-scale QP problem into a series of small QP sub-problems. (Arguably, this can also be regarded as a data-processing approach.) Each sub-problem uses a small set of selected training instances, called the *working set*. The decomposition-based method repeatedly solves a small-size QP problem until it converges. SVMLight (Joachims, 1998) is a representative decomposition algorithm, which allows a user to set the working-set size. The *Sequential Minimization Optimization* (SMO) (Platt, 1998) can be regarded as a special case where the working-set size is set to be two. Two widely used implementations of SMO are LibSVMs (Chang & Lin, 2001) and SVMTorch (Collobert & Bengio, 2001). The decomposition still has to solve a QP problem, though its size is much smaller. Current implementations resort to the primal-dual interior-point method (IPMs) (Mehrotra, 1992) to solve the QP. In SMO, however, the QP sub-problem with only two instances can be analytically solved without utilizing the primal-dual IPM.

The decomposition-based method enjoys the advantage of small usage of main memory and hence can be employed for training large-scale datasets. Their computational cost, however, remains high, about $\mathcal{O}(n^2)$. The total computational cost is mainly determined by the number of iterations to converge and the cost of working on a QP sub-problem at each iteration. Two factors, the size of the working set and the number of iterations, trade off with each other. When a smaller fraction of instances are selected as the working set, the decomposition-based method takes a larger number of iterations to converge. These iterations, unfortunately, cannot be executed in parallel because of data dependency between iterations. Therefore, the computational cost of the decomposition-based method is difficult to improve further, even when a large number of machines are available. In contrast to the decomposition-based method, our proposed PSVM reduces the size of the QP problem via approximate factorization. In addition, PSVM parallelizes both its memory use and computation, and therefore its performance can scale linearly with the number of machines.

Factorization-based method has been proposed before. Traditional matrix factorization algorithms such as LU, QR, and Cholesky suffer from high memory requirement and qubic computational intensity. Approximate factorization algorithms such as Kronecker (Wu et al., 2005) and Incomplete Cholesky factorization[1] (ICF) (Fine & Scheinberg,

[1]There are two kinds of ICF found in literatures. The other

2001; Bach & Jordan, 2005) can reduce computational cost, but the initial memory requirement is $O(np)$, and hence not practical for very large data set. PSVM devises parallel ICF (PICF) as its initial step, which loads training instances onto parallel machines and performs factorization simultaneously on these machines. The key "trick" is that PICF performs row-based factorization, rather than the traditional column-based, to achieve both data and computation parallelization.

Several recent works (e.g., (Lee & Mangasarian, 2001; Tsang et al., 2005; Joachims, 2006)) attempt to speed up SVMs. These works encounter at least one of the following three shortcomings, which cap their usability and scalability: large memory requirement, unable to work with kernel functions, or not parallelizable. More specifically, the work of (Joachims, 2006) though it can train linear SVMs in linear time, requires large memory and cannot work with kernel functions (as the title "linear SVMs" indicates).

3. PSVM Algorithm

The key step of PSVM is parallel ICF (PICF). Once PICF has loaded n training data distributedly on m machines, and reduced the size of the kernel matrix through factorization, IMP can be solved on parallel machines simultaneously. We present PICF first, and then describe how IPM takes advantage of PICF.

3.1. Parallel ICF (PICF)

ICF can approximate Q ($Q \in R^{n \times n}$) by a smaller matrix H ($H \in R^{n \times p}, p \ll n$), i.e., $Q \approx HH^T$. ICF, together with SMW (the *Sherman-Morrison-Woodbury formula*), can greatly reduce the computational complexity in solving an $n \times n$ linear system. Specifically for SVMs, ICF and SMW can speed up the Interior Point Method (IPM) from $\mathcal{O}(n^3)$ to $\mathcal{O}(np^2)$. However, the $\mathcal{O}(np)$ memory use of ICF for loading Q continues to plague large-scale training tasks. Our row-based parallel ICF (PICF) distributes memory use on m machines, and hence reduces memory use to $\mathcal{O}(np/m)$.

Let vector $\mathbf{v}$ be the diagonal of Q and suppose the pivots are $\{i_1, i_2, \ldots, i_k\}$, the kth iteration of ICF computes three equations:

$$H(i_k, k) = \sqrt{\mathbf{v}(i_k)} \quad (3)$$

$$H(J_k, k) = (Q(J_k, k) - \sum_{j=1}^{k-1} H(J_k, j)H(i_k, j))/H(i_k, k) \quad (4)$$

$$\mathbf{v}(J_k) = \mathbf{v}(J_k) - H(J_k, k)^2, \quad (5)$$

where J_k denotes the complement of $\{i_1, i_2, \ldots, i_k\}$. The algorithm iterates until the approximation of Q by $H_k H_k^T$ (measured by $trace(Q - H_k H_k^T)$) is satisfactory, or the predefined maximum iterations (or say, the desired rank of the ICF matrix) p is reached.

As suggested by G. Golub, a parallelized ICF algorithm can be obtained by constraining the parallelized Cholesky Factorization algorithm, iterating at most p times. Unfortunately, in the proposed algorithm (Golub & Loan, 1996), matrix H is distributed by columns in a round-robin way on m machines (hence we call it column-based parallelized ICF). Such column-based approach cannot gain full benefit from parallelization for two major reasons:

1. Large storage requirement. All training data are needed for each machine to calculate $Q(J_k, k)$. Therefore, each machine must be able to store a local copy of the training data.

2. Limited parallelizable computation. Only the inner product calculation ($\sum_{j=1}^{k-1} H(J_k, j)H(i_k, j)$) in (4) can be parallelized. The calculation of pivot selection, the summation of local inner product result, column calculation in (4), and the vector update in (5) must be performed on one single machine.

To remedy these shortcomings of the column-based approach, we propose a row-based approach to parallelize ICF, which we summarize in Algorithm 1. Our row-based approach starts by initializing variables and loading training data onto m machines in a round-robin fashion (Steps 1 to 5). The algorithm then performs the ICF main loop until the termination criteria are satisfied (e.g., the rank of matrix H reaches p). In the main loop, PICF performs five tasks in each iteration k:

1. Distributedly find a pivot, which is the largest value in the diagonal $\mathbf{v}$ of matrix Q (steps 7 to 10). Notice that PICP computes only needed elements in Q from training data, and it does not store Q.

2. Set the machine where the pivot resides as the *master* (step 11).

3. On the *master*, PICF calculates $H(i_k, k)$ according to (3) (step 12).

kind (Benzi, 2002) is often used in a linear-system solver as its preconditioner, which approximates sparse matrix Q ($Q \in R^{n \times n}$) by a sparse lower-triangular matrix S ($S \in R^{n \times n}$), i.e. $Q \approx SS^T$. This kind of ICF deals only with sparse matrices, and is not applicable to our purpose of speeding up SVMs.

4. The *master* then broadcasts the pivot instance $\mathbf{x}_{i_k}$ and the pivot row $H(i_k, :)$ (step 13).

5. Distributedly compute (4) and (5) (steps 14 and 15).

At the end of the algorithm, H is stored distributedly on m machines, ready for parallel IPM (presented in the next section). PICF enjoys three advantages: parallel memory use ($\mathcal{O}(np/m)$), parallel computation ($\mathcal{O}(p^2n/m)$), and low communication overhead. Particularly on the communication overhead, its fraction of the entire computation time shrinks as the problem size grows. We will verify this in the evaluation section. This pattern permits a larger problem to be solved on more machines to take advantage of parallel memory use and computation.

3.2. Parallel IPM (PIPM)

As mentioned in Section 1, the most effective algorithm to solve a constrained QP problem is the primal-dual IPM. For detailed description of IPM, please consult (Boyd, 2004; Mehrotra, 1992). For the purpose of SVM training, IPM boils down to solving the following equations in the Newton step iteratively.

$$\triangle\boldsymbol{\lambda} = -\boldsymbol{\lambda} + \mathrm{vec}\left(\frac{1}{t(C-\alpha_i)}\right) + \mathrm{diag}(\frac{\lambda_i}{C-\alpha_i})\triangle\mathbf{x} \tag{6}$$

$$\triangle\boldsymbol{\xi} = -\boldsymbol{\xi} + \mathrm{vec}\left(\frac{1}{t\alpha_i}\right) - \mathrm{diag}(\frac{\xi_i}{\alpha_i})\triangle\mathbf{x} \tag{7}$$

$$\triangle\nu = \frac{\mathbf{y}^T\boldsymbol{\Sigma}^{-1}\mathbf{z} + \mathbf{y}^T\boldsymbol{\alpha}}{\mathbf{y}^T\boldsymbol{\Sigma}^{-1}\mathbf{y}} \tag{8}$$

$$D = diag(\frac{\xi_i}{\alpha_i} + \frac{\lambda_i}{C-\alpha_i}) \tag{9}$$

$$\triangle\mathbf{x} = \boldsymbol{\Sigma}^{-1}(\mathbf{z} - \mathbf{y}\triangle\nu), \tag{10}$$

where $\boldsymbol{\Sigma}$ and $\mathbf{z}$ depend only on $[\boldsymbol{\alpha}, \boldsymbol{\lambda}, \boldsymbol{\xi}, \boldsymbol{\nu}]$ from the last iteration as follows:

$$\boldsymbol{\Sigma} = \mathbf{Q} + \mathbf{diag}(\frac{\xi_i}{\alpha_i} + \frac{\lambda_i}{C-\alpha_i}) \tag{11}$$

$$\mathbf{z} = -\mathbf{Q}\boldsymbol{\alpha} + \mathbf{1}_n - \nu\mathbf{y} + \frac{1}{t}\mathrm{vec}(\frac{1}{\alpha_i} - \frac{1}{C-\alpha_i}). \tag{12}$$

The computation bottleneck is on matrix inverse, which takes place on $\boldsymbol{\Sigma}$ for solving $\triangle\nu$ in (8) and $\triangle\mathbf{x}$ in (10). Equation (11) shows that $\boldsymbol{\Sigma}$ depends on Q, and we have shown that Q can be approximated through PICF by HH^T. Therefore, the bottleneck of the Newton step can be sped up from $\mathcal{O}(n^2)$ to $\mathcal{O}(p^2n)$, and be parallelized to $\mathcal{O}(p^2n/m)$.

Distributed Data Loading

To minimize both storage and communication cost, PIPM stores data distributedly as follows:

Algorithm 1 Row-based PICF

Input: n training instances; p: rank of ICF matrix H; m: number of machines
Output: H distributed on m machines
Variables:
$\mathbf{v}$: fraction of the diagonal vector of Q that resides in local machine
k: iteration number; STATE $\mathbf{x}_i$: the ith training instance
M: machine index set, $M = \{0, 1, \ldots, m-1\}$
I_c: row-index set on machine c ($c \in M$), $I_c = \{c, c+m, c+2m, \ldots\}$

1: for $i = 0$ to $n-1$ do
2: Load $\mathbf{x}_i$ into machine $i\%m$.
3: end for
4: $k \leftarrow 0$; $H \leftarrow 0$; $\mathbf{v} \leftarrow$ the fraction of the diagonal vector of Q that resides in local machine. ($\mathbf{v}(i)(i \in I_m)$ can be obtained from $\mathbf{x}_i$)
5: Initialize *master* to be machine 0.
6: while $k < p$ do
7: Each machine $c \in M$ selects its local pivot value, which is the largest element in $\mathbf{v}$:

$$\mathbf{lpv}_{k,c} = \max_{i \in I_c} \mathbf{v}(i)$$

. and records the local pivot index, the row index corresponds to $\mathbf{lpv}_{k,c}$:

$$\mathbf{lpi}_{k,c} = arg\max_{i \in I_c} \mathbf{v}(i)$$

.
8: Gather $\mathbf{lpv}_{k,c}$'s and $\mathbf{lpi}_{k,c}$'s ($c \in M$) to *master*.
9: The *master* selects the largest local pivot value as global pivot value $\mathbf{gpv}_k$ and records in i_k, row index corresponding to the global pivot value.

$$\mathbf{gpv}_k = \max_{c \in M} \mathbf{lpv}_{k,c}$$

.
10: The *master* broadcasts $\mathbf{gpv}_k$ and i_k.
11: Change *master* to machine $i_k\%m$.
12: Calculate $H(i_k, k)$ according to (3) on *master*.
13: The *master* broadcasts the pivot instance $\mathbf{x}_{i_k}$ and the pivot row $H(i_k, :)$. (Only the first $k+1$ values of the pivot row need to be broadcast, since the remainder are zeros.)
14: Each machine $c \in M$ calculates its part of the kth column of H according to (4).
15: Each machine $c \in M$ updates $\mathbf{v}$ according to (5).
16: $k \leftarrow k + 1$
17: end while

- *Distribute matrix data*. H is distributedly stored at the end of PICF.
- *Distribute* $n \times 1$ *vector data*. All $n \times 1$ vectors are distributed in a round-robin fashion on m machines. These vectors are $\mathbf{z}$, $\boldsymbol{\alpha}$, $\boldsymbol{\xi}$, $\boldsymbol{\lambda}$ $\Delta\mathbf{z}$, $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{\xi}$, and $\Delta\boldsymbol{\lambda}$.
- *Replicate global scalar data*. Every machine caches a copy of global data including ν, t, n, and $\Delta\nu$. Whenever a scalar is changed, a broadcast is required to maintain global consistency.

Parallel Computation of $\triangle\nu$

Rather than walking through all equations, we describe how PIMP solves (8), where Σ^{-1} appears twice. An interesting observation is that parallelizing $\Sigma^{-1}z$ (or $\Sigma^{-1}y$) is simpler than parallelizing Σ^{-1}. Let us explain how parallelizing $\Sigma^{-1}z$ works, and parallelizing $\Sigma^{-1}y$ can follow suit.

According to SMW (the *Sherman-Morrison-Woodbury formula*), we can write $\Sigma^{-1}z$ as

$$\begin{aligned}\Sigma^{-1}z &= (D+Q)^{-1} \approx (D+HH^T)^{-1} \\ &= D^{-1}z - D^{-1}H(I+H^TD^{-1}H)^{-1}H^TD^{-1}z \\ &= D^{-1}z - D^{-1}H(GG^T)^{-1}H^TD^{-1}z.\end{aligned}$$

$\Sigma^{-1}z$ can be computed in four steps:

1. Compute $D^{-1}z$.

   D can be derived from locally stored vectors, following (9). $D^{-1}z$ is a $n \times 1$ vector, and can be computed locally on each of the m machines.

2. Compute $t_1 = H^TD^{-1}z$.

   Every machine stores some rows of H and their corresponding part of $D^{-1}z$. This step can be computed locally on each machine. The results are sent to the *master* (which can be any machine for PIPM) to aggregate into t_1 for the next step.

3. Compute $(GG^T)^{-1}t_1$.

   This step is completed on the *master*, since it has all the required data. G is a $p \times p$ matrix, which is derived from H by SMW. Computing $t_2 = (GG^T)^{-1}t_1$ is equivalent to solving the linear equation system $t_1 = (GG^T)t_2$. PIPM first solves $t_1 = Gy_0$, then $y_0 = G^Tt2$. Once it has obtained y_0, PIPM can solve $G^Tt_2 = y_0$ to obtain t_2. The *master* then broadcasts t_2 to all machines.

4. Compute $D^{-1}Ht_2$

   All machines have a copy of t_2, and can compute $D^{-1}Ht_2$ locally to solve for $\Sigma^{-1}z$.

Similarly, $\Sigma^{-1}y$ can be computed at the same time. Once we have obtained both, we can solve $\Delta\nu$ according to (8).

3.3. Parallel Computing b and Writing Back

When the IPM iteration stops, we have the value of α and hence the classification function

$$f(x) = \sum_{i=1}^{N_s} \alpha_i y_i K(\mathbf{s}_i, x) + b$$

.

Here, N_s is the number of support vectors and $\mathbf{s}_i$'s are support vectors. In order to complete this classification function, b must be computed. According to the SVM model, given a support vector $\mathbf{s}$, we obtain one of the two results for $f(\mathbf{s})$: $f(\mathbf{s}) = +1$, if $y_s = +1$, or $f(\mathbf{s}) = -1$, if $y_s = -1$. In practice, we can select M, say $1,000$, support vectors and compute the average of the b_ss.

$$b = \frac{1}{M}\sum_{j=1}^{M}(y_{\mathbf{s}_j} - \sum_{i=1}^{N_s}\alpha_i y_i K(\mathbf{s}_i, \mathbf{s}_j))$$

Since the support vectors are distributed on m machines, PSVM collects them in parallel to compute b. For this purpose, we transform the above formula into the following:

$$b = \frac{1}{M}\sum_{j=1}^{M} y_{\mathbf{s}_j} - \frac{1}{M}\sum_{i=1}^{N_s}\alpha_i y_i \sum_{j=1}^{M} K(\mathbf{s}_i, \mathbf{s}_j)$$

The M support vectors and their labels y are first broadcast to all machines. All m machines then compute their local results. Finally, the local results are summed up by a reduce operation (Dean & Ghemawat, 2004). When b has been computed, the last task of PSVM is to store the model file on a distributed file system for later classification use.

4. Empirical Evaluations

In this section, we report the experimental results for both the effectiveness and efficiency of PSVM. Our experiments were designed to answer the following questions:

1. How well does PSVM perform on a classification task in terms of class-prediction accuracy?
2. How does PSVM scale along with the larger training set and more machines, in terms of training time and per-machine memory usage?
3. What is the fraction of the non-parallelizable part (communication overhead) of PSVM, and how does this affect scalability?

Table 1. The Performance of PSVM on Reuters.

Positive Precision	0.7690
Positive Recall	0.9391
Negative Precision	0.9453
Negative Recall	0.7896
Overall Accuracy	0.8535

4.1. Experiment setup

We performed evaluations on two datasets. We used a 120k, binary-class image dataset to evaluate classification accuracy. The reason for our selecting this "small" dataset is because the competing LibSVM does not scale well with large datasets. The second dataset we used is the 780k Reuters RCV1 collection[2], a community standard corpus to be the training and testing data. We sampled this dataset to create up to a 1m training dataset (with replication) to test the scalability of PSVM.

The experiments were performed on as many as twenty Xeox workstations with Intel Pentium 4 3.00GHz CPU, 2GB RAM with 16KB L1 cache and 2MB L2 cache, running Linux.

4.2. Classification Accuracy

There are two matters we would like to examine. The first is how the approximate nature of PSVM affects class-prediction accuracy. The second is how the parallel nature of PSVM affects class-prediction accuracy. The study of (Wu et al., 2006) has shown that ICP (not PICP) does not cause class-prediction to degrade even when p is about $\sqrt{n}$. In our study, we added the parallel factor into our experiments.

In the first experiment, we used the 120k image dataset by splitting the data $90\%/10\%$ for training and testing. We ran PSVM (with p set to 600) and LibSVM[3] on the training data, and compared their class-prediction accuracy on the testing data. We used Laplacian RBF kernel for LibSVM. All parameters were carefully tuned ($C = 2$ and $\gamma = 0.00125$) via cross-validation. It took seven hours for LibSVM to complete training on one machine, whereas PSVM took 40 minutes on seven machines. (The speedup is more than linear because of the approximate nature of PSVM.) Figure 1 compares the precision/recall curves of class-prediction. Figure 2 displays the ROC curves of the two software systems. Both figures show that the class-prediction accuracy of PSVM and LibSVM are virtually tied.

[2]http://jmlr.csail.mit.edu/papers/volume5/lewis04a/lyrl2004_rcv1v2_README.htm

[3]http://www.csie.ntu.edu.tw/ cjlin/libsvm/

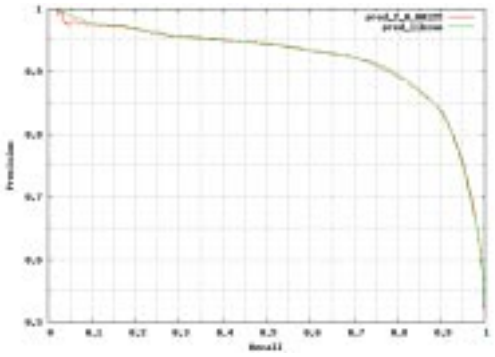


Figure 1. Precision-recall Curves of PSVM and LibSVM.

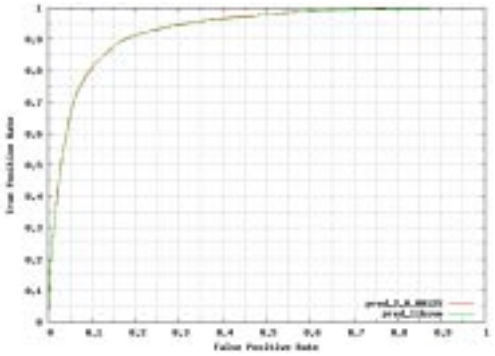


Figure 2. ROC Curves of PSVM and LibSVM.

Our second test examined whether the number of machines can affect class-prediction accuracy. Due to roundings of intermediate results, PSVM run on different number of machines can produce slightly different results. We sampled 250k training data and 100k testing data from the Reuters RCV1 collection. We ran PSVMs on one, two, three, four, and five machines, respectively. We set the hyperparameter C as 1.0, the RBF kernel σ as 1.0, and PICF rank p as 250, which is less than $\sqrt{n}$. Table 1 shows that on all five tests, the class-prediction accuracy is virtually the same. The results did appear to have negligible differences, which did not affect accuracy.

4.3. The scalability of the PSVM

For the scalability evaluation, we randomly sampled 100k articles from the Reuters RCV1 collection as testing data, while using the rest of 780k articles as training data. The number of unique features was $47,236$ with the sparsity of 0.16%. Based on the 780k training examples, we constructed four training sets by random sampling with replication, which contained 250k, 500k, 750k, and 1m training examples, respectively. Among the 100k testing samples, there were $42,715$ positive samples and $57,285$ negative

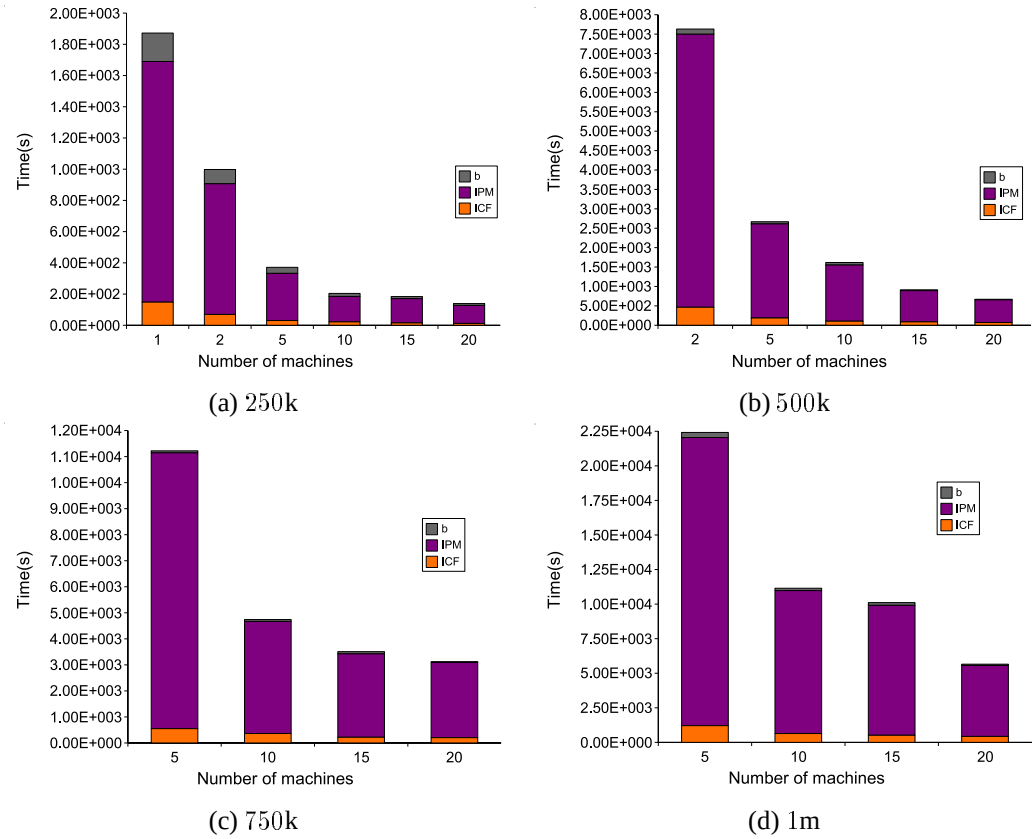


Figure 3. Training Time of Different n and m.

ones. Since we mainly focus on testing the scalability of PSVM, we did not perform cross-validation.

We conducted training with training sets 250k, 500k, 750k and 1m, each on 1, 2, 5, 10, 15 and 20 machines. The training time mainly comprised the time for performing PICF, PIPM and, b estimation. We drew four bar graphs in Figure 3 to depict the composition of training time and the training time versus number of machines. Figure 3 illustrates that the total training time is inversely proportional to the number of machines. In other words, we gain linear reduction in training time when adding more machines. Note that not all training time is available in the figure, because some datasets took too long to finish on a couple of machines. This gives us a strong indication that PSVM not only significantly accelerates the training process, but more significantly, it can train large datasets. Examining closely at the three major computation components—PICF, PIPM and computing b—we can see that the PIPM step takes most of the training time, and it can be well parallelized as described in Section 3.

We list per-machine memory versus number of machines involved in training in Table 2 for four training datasets. The memory requirement for each machine is significantly reduced given more machines. When using only one machine, it is not likely for the machine to accommodate over 500k training data, whereas each machine takes 490MB memory when using 20 machines, with the 1m training set. PSVM scales well in both memory use and computational time.

Table 2. Per-machine Memory Usage (MB).

$m \backslash n$	250k	500k	750k	1000k
1	839	N/A	N/A	N/A
2	419	1295	N/A	N/A
5	186	535	1428	1823
10	99	290	586	942
15	79	223	387	669
20	57	156	316	490

4.4. Communication Overhead

PSVM requires synchronization among computers. The synchronization is conducted through message passing. Thus this communication overhead must be examined to understand the effectiveness of PSVM.

Table 3. The Ratio of Overhead.

$m\backslash n$	250k	500k	750k	1000k
5	0.009	0.006	0.005	0.002
10	0.016	0.016	0.006	0.005
15	0.027	0.012	0.008	0.009
20	0.049	0.032	0.023	0.014

We use the ratio of communication time over computational time to calculate overhead. Table 3 lists the ratios of training on the four datasets, given increasing numbers of machines. The first observation is that communication time is relatively low compared to computational time. When more machines are added, the ratios increase, which is understandable by the Amdahl's law. Fortunately, the ratios decrease when the dataset size grows. This is because the communication complexity is $O(p^2)$, whereas the computational complexity is $O(p^2n/m)$. As a result, we can use more machines to train a larger dataset. Also, we can employ more machines to speed up training as the dataset size increases.

5. Conclusion

In this paper, we have shown how SVMs can be parallelized to achieve scalable performance. PSVM distributedly loads training data on parallel machines, reducing memory requirement through approximate factorization on the kernel matrix. PSVM solves IPM in parallel by cleverly arranging computation order. Through empirical studies, we have shown that PSVM does not sacrifice class-prediction accuracy for scalability, and it scales well with training data size.

PSVM is now being used by several groups to conduct training on massive amounts of data. Our future work will focus on making PSVM fail-safe. We are currently investigating a similar technique employed by (Dean & Ghemawat, 2004) to achieve the fail-safe objective.

References

Aizerman, M. A., Braverman, E. M., & Rozonoer, L. I. (1964). Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, *25*, 821–837.

Bach, F. R., & Jordan, M. I. (2005). Predictive low-rank decomposition for kernel methods. *Proceedings of the 22nd International Conference on Machine Learning*.

Benzi, M. (2002). Preconditioning techniques for large linear systems: A survey. *Journal of Computational Physics*, *182*, 418–477.

Boyd, S. (2004). *Convex optimization*. Cambridge University Press.

Chang, C.-C., & Lin, C.-J. (2001). *LIBSVM: a library for support vector machines*. Software available at `http://www.csie.ntu.edu.tw/ cjlin/libsvm`.

Collobert, R., & Bengio, S. (2001). Svmtorch: support vector machines for large-scale regression problems. *Journal of Machine Learning Research*, *1*, 143–160.

Courant, R., & Hilbert, D. (1953). *Method of mathematical physics*, vol. 1. Interscience Publishers.

Dean, J., & Ghemawat, S. (2004). Mapreduce: Simplified data processing on large clusters. *OSDI'04: Symposium on Operating System Design and Implementation*.

Fine, S., & Scheinberg, K. (2001). Efficient svm training using low-rank kernel representations. *Journal of Machine Learning Research*, *2*, 243–264.

Golub, G. H., & Loan, C. F. V. (1996). *Matrix computations*. Johns Hopkins University Press.

Joachims, T. (1998). Making large-scale svm learning practical. *Advances in Kernel Methods - Support Vector Learning*.

Joachims, T. (2006). Training linear svms in linear time. *ACM KDD*, 217–2226.

Lee, Y.-J., & Mangasarian, O. L. (2001). Rsvm: Reduced support vector machines. *First SIAM International Conference on Data Mining*. Chicago.

Mehrotra, S. (1992). On the implementation of a primal-dual interior point method. *SIAM J. Optimization*, *2*, 575–601.

Platt, J. (1998). *Sequential minimal optimization: A fast algorithm for training support vector machines* (Technical Report MSR-TR-98-14). Microsoft Research.

Tsang, I. W., Kwok, J. T., & Cheung, P.-M. (2005). Core vector machines: Fast svm training on very large data sets. *Journal of Machine Learning Research*, *6*, 363–392.

Vapnik, V. (1995). *The nature of statistical learning theory*. New York: Springer.

Wu, G., Chang, E. Y., Chen, Y.-K., & Hughes, C. (2006). Incremental approximate matrix factorization for speeding up support vector machines. *ACM KDD*.

Wu, G., Zhang, Z., & Chang, E. Y. (2005). Kronecker factorization for speeding up kernel machines. *SIAM International Conference on Data Mining*.